SIMATS ENGINEERING

CO1-ASSESSMENT TOOL 1-Constrain Based Problem

COURSE NAME: R PROGRAMMING COURSE CODE: ITA04

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.

(Bloom Level -BL2- BL4)

Assessment Tool Weightage: 20%

Code of Conduct:

*I **YUGITHA B (192424102)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Student Mark Analysis using Vectors

```
marks <- c(45, 67, 82, 56, 91, 73, 48, 65, 88, 52)
print("Marks of students:")
print(marks)
total <- sum(marks)
average <- mean(marks)
print(paste("Total =", total))
print(paste("Average =", average))
print(paste("Maximum mark =", max(marks)))
print(paste("Minimum mark =", min(marks)))
print("Marks greater than 50:")
print(marks[marks > 50])
position <- which.max(marks)
print(paste("Position of highest mark =", position))
```

```
> marks <- c(45, 67, 82, 56, 91, 73, 48, 65, 88, 52)
> print("Marks of students:")
[1] "Marks of students:"
> print(marks)
 [1] 45 67 82 56 91 73 48 65 88 52
> total <- sum(marks)
> average <- mean(marks)
> print(paste("Total =", total))
[1] "Total = 667"
> print(paste("Average =", average))
[1] "Average = 66.7"
> print(paste("Maximum mark =", max(marks)))
[1] "Maximum mark = 91"
> print(paste("Minimum mark =", min(marks)))
[1] "Minimum mark = 45"
> print("Marks greater than 50:")
[1] "Marks greater than 50:"
> print(marks[marks > 50])
[1] 67 82 56 91 73 65 88 52
> position <- which.max(marks)
> print(paste("Position of highest mark =", position))
[1] "Position of highest mark = 5"
```

2. Student Information using Different Data Types

```
student_name <- "Yugitha"
register_number <- 101
age <- 20L
```

```
gender <- "Female"
marks <- 85.5
status <- TRUE
print(student_name)
print(register_number)
print(age)
print(gender)
print(marks)
print(status)
print(typeof(student_name))
print(typeof(register_number))
print(typeof(age))
print(typeof(gender))
print(typeof(marks))
print(typeof(status))
```

```
> print(student_name)
[1] "Yugitha"
> print(register_number)
[1] 101
> print(age)
[1] 20
> print(gender)
[1] "Female"
> print(marks)
[1] 85.5
> print(status)
[1] TRUE
> print(typeof(student_name))
[1] "character"
> print(typeof(register_number))
[1] "double"
> print(typeof(age))
[1] "integer"
> print(typeof(gender))
[1] "character"
> print(typeof(marks))
[1] "double"
> print(typeof(status))
[1] "logical"
```

3. Matrix Operations in R

```
A <- matrix(c(10, 20, 30, 40), nrow = 2, ncol = 2)
B <- matrix(c(5, 10, 15, 20), nrow = 2, ncol = 2)
print("Matrix A:")
print(A)
print("Matrix B:")
print(B)
print("Addition:")
print(A + B)
print("Subtraction:")
print(A - B)
print("Multiplication:")
print(A %*% B)
print("Transpose of A:")
print(t(A))
print("Element at row 1, column 2:")
print(A[1, 2])
print("First row of A:")
print(A[1, ])
```

```
> print("Addition:")
[1] "Addition:"
> print(A + B)
     [,1] [,2]
[1,]   15   45
[2,]   30   60
> print("Subtraction:")
[1] "Subtraction:"
> print(A - B)
     [,1] [,2]
[1,]    5   15
[2,]   10   20
> print("Multiplication:")
[1] "Multiplication:"
> print(A %*% B)
     [,1] [,2]
[1,]  350  750
[2,]  500 1100
> print("Transpose of A:")
[1] "Transpose of A:"
> print(t(A))
     [,1] [,2]
[1,]   10   20
[2,]   30   40
> print("Element at row 1, column 2:")
[1] "Element at row 1, column 2:"
> print(A[1, 2])
[1] 30
> print("First row of A:")
[1] "First row of A:"
> print(A[1, ])
[1] 10 30
```

4. Employee Salary Analysis using Lists

```
employee <- list(
  Name = "Arun",
  ID = 101,
  Department = "IT",
  BasicSalary = 30000,
  Allowances = 5000,
  Deductions = 2000
)
gross_salary <- employee$BasicSalary + employee$Allowances
net_salary <- gross_salary - employee$Deductions
print(employee)
print(paste("Gross Salary =", gross_salary))
print(paste("Net Salary =", net_salary))
```

```
$Name
[1] "Arun"

$ID
[1] 101

$Department
[1] "IT"

$BasicSalary
[1] 30000

$Allowances
[1] 5000

$Deductions
[1] 2000

> print(paste("Gross Salary =", gross_salary))
[1] "Gross Salary = 35000"
> print(paste("Net Salary =", net_salary))
[1] "Net Salary = 33000"
```

5. Product Sales using 3D Array

```
sales <- array(
  c(100, 120, 150,
    80, 90, 110,
    130, 140, 160,
    70, 85, 95),
```

```
  dim = c(3, 2, 2)
)
dimnames(sales) <- list(
  Product = c("Product A", "Product B", "Product C"),
  Branch = c("Branch 1", "Branch 2"),
  Month = c("January", "February")
)
print("Sales Array:")
print(sales)
print("Sales of Product A in Branch 1 in January:")
print(sales["Product A", "Branch 1", "January"])
total_sales <- sum(sales)
print(paste("Total Sales =", total_sales))
```

```
> print(sales)
, , Month = January

           Branch
Product     Branch 1 Branch 2
  Product A      100       80
  Product B      120       90
  Product C      150      110

, , Month = February

           Branch
Product     Branch 1 Branch 2
  Product A      130       70
  Product B      140       85
  Product C      160       95

> print("Sales of Product A in Branch 1 in January:")
[1] "Sales of Product A in Branch 1 in January:"
> print(sales["Product A", "Branch 1", "January"])
[1] 100
> total_sales <- sum(sales)
> print(paste("Total Sales =", total_sales))
[1] "Total Sales = 1330"
```

6. Student Grade Analysis using Factors

```
grades <- factor(c(
  "A", "B", "A", "C", "B",
  "A", "D", "B", "C", "A"
))
print("Grades:")
print(grades)
```

```
print("Factor Levels:")
print(levels(grades))
print("Number of students in each grade:")
print(table(grades))
print("Grade of first student:")
print(grades[1])
print("Grade of fifth student:")
print(grades[5])
```

```
> print("Grades:")
[1] "Grades:"
> print(grades)
 [1] A B A C B A D B C A
Levels: A B C D
> print("Factor Levels:")
[1] "Factor Levels:"
> print(levels(grades))
[1] "A" "B" "C" "D"
> print("Number of students in each grade:")
[1] "Number of students in each grade:"
> print(table(grades))
grades
A B C D
4 3 2 1
> print("Grade of first student:")
[1] "Grade of first student:"
> print(grades[1])
[1] A
Levels: A B C D
> print("Grade of fifth student:")
[1] "Grade of fifth student:"
> print(grades[5])
[1] B
Levels: A B C D
```

7. Student Data Frame Problem

```
student_data <- data.frame(
  Student_ID = c(101, 102, 103, 104, 105),
  Name = c("Arun", "Bala", "Divya", "Kavi", "Ravi"),
  Age = c(20, 21, 20, 19, 21),
  Department = c("AI&DS", "CSE", "AI&DS", "ECE", "CSE"),
  Gender = c("M", "M", "F", "F", "M"),
  Marks = c(85, 72, 90, 68, 78)
)
print(student_data)
```

```
print("Student Names:")
print(student_data$Name)
print("First student:")
print(student_data[1, ])
print(paste("Average Marks =", mean(student_data$Marks)))
print(paste("Maximum Marks =", max(student_data$Marks)))
print(paste("Minimum Marks =", min(student_data$Marks)))
```

```
> print("Student Names:")
[1] "Student Names:"
> print(student_data$Name)
[1] "Arun"  "Bala"  "Divya" "Kavi"  "Ravi"
> print("First student:")
[1] "First student:"
> print(student_data[1, ])
  Student_ID Name Age Department Gender Marks
1        101 Arun  20      AI&DS      M    85
> print(paste("Average Marks =", mean(student_data$Marks)))
[1] "Average Marks = 78.6"
> print(paste("Maximum Marks =", max(student_data$Marks)))
[1] "Maximum Marks = 90"
> print(paste("Minimum Marks =", min(student_data$Marks)))
[1] "Minimum Marks = 68"
>
```

8. Logical Vector-Based Student Selection

```
marks <- c(85, 45, 72, 91, 38, 65, 78, 49,
           56, 82, 33, 70, 95, 42, 60)
above_75 <- marks[marks > 75]
between_50_75 <- marks[marks >= 50 & marks <= 75]
below_50 <- marks[marks < 50]
passed <- marks[marks >= 50]
print("Marks above 75:")
print(above_75)
print("Marks between 50 and 75:")
print(between_50_75)
print("Marks below 50:")
print(below_50)
```

```
print("Passed students:")
print(passed)
```

```
[1] "Marks above 75:"
> print(above_75)
[1] 85 91 78 82 95
> print("Marks between 50 and 75:")
[1] "Marks between 50 and 75:"
> print(between_50_75)
[1] 72 65 56 70 60
> print("Marks below 50:")
[1] "Marks below 50:"
> print(below_50)
[1] 45 38 49 33 42
> print("Passed students:")
[1] "Passed students:"
> print(passed)
 [1] 85 72 91 65 78 56 82 70 95 60
```

9. R Data Structure Problem

```
marks <- c(80, 75, 90, 65)
print("Vector:")
print(marks)
print(marks[1])
student <- list(
  Name = "Yugitha",
  Age = 20,
  Marks = 85
)
print("List:")
print(student)
print(student$Name)
m <- matrix(c(10, 20, 30, 40), nrow = 2)
print("Matrix:")
print(m)
print(m[1, 2])
grade <- factor(c("A", "B", "A", "C"))
print("Factor:")
```

```
print(grade)
print(levels(grade))
data <- data.frame(
  ID = c(101, 102, 103),
  Name = c("Arun", "Bala", "Divya"),
  Marks = c(80, 75, 90)
)
print("Data Frame:")
print(data)
print("Names:")
print(data$Name)
```

```
[1] "Marks above 75:"
> print(above_75)
[1] 85 91 78 82 95
> print("Marks between 50 and 75:")
[1] "Marks between 50 and 75:"
> print(between_50_75)
[1] 72 65 56 70 60
> print("Marks below 50:")
[1] "Marks below 50:"
> print(below_50)
[1] 45 38 49 33 42
> print("Passed students:")
[1] "Passed students:"
> print(passed)
 [1] 85 72 91 65 78 56 82 70 95 60
```

10. Student Result Processing Application

```
names <- c("Arun", "Bala", "Divya", "Kavi", "Ravi")
maths <- c(80, 65, 90, 55, 72)
science <- c(75, 70, 85, 60, 68)
computer <- c(88, 60, 92, 58, 75)
total <- maths + science + computer
average <- total / 3
grade <- factor(
  ifelse(average >= 80, "A",
         ifelse(average >= 70, "B",
```

```
            ifelse(average >= 60, "C", "D")))
)
result_matrix <- cbind(
  Maths = maths,
  Science = science,
  Computer = computer,
  Total = total,
  Average = average
)
print("Result Matrix:")
print(result_matrix)
student_result <- data.frame(
  Name = names,
  Maths = maths,
  Science = science,
  Computer = computer,
  Total = total,
  Average = average,
  Grade = grade
)
print("Final Student Result:")
print(student_result)
```

```
> print("Final Student Result:")
[1] "Final Student Result:"
> print(student_result)
   Name Maths Science Computer Total  Average Grade
1  Arun    80      75       88   243 81.00000     A
2  Bala    65      70       60   195 65.00000     C
3 Divya    90      85       92   267 89.00000     A
4  Kavi    55      60       58   173 57.66667     D
5  Ravi    72      68       75   215 71.66667     B
>
```